Customer Behavior Analytics using Python, SQL & Power BI

1. Project Overview

This project presents an end-to-end data analytics solution using Python, SQL, and Power BI to analyze customer shopping behavior.

The objective is to identify purchasing patterns, understand customer segments, and generate actionable insights that support business decision-making and revenue growth.

2. Dataset Summary

- Rows: 3,900
- Columns: 18
- Key Features :
  #Customer demographics (Age, Gender, Location, Subscription Status)
  #Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
  #Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases
  #Frequency of Purchases, Review Rating, Shipping Type)
  #Missing Data: 37 values in Review Rating column

3. Business Objective

The company aims to:

- Understand customer spending behavior
- Identify high-value and repeat customers
- Analyze product and category performance
- Evaluate the impact of subscriptions and discounts

4. Data Workflow

Step 1 – Python (Data Preparation)

- Data cleaning and preprocessing using Pandas
- Handling missing values (Review Rating)
- Feature engineering (Age Groups, Purchase Frequency)

Step 2 – SQL (Data Analysis)

- Data loaded into PostgreSQL
- Business queries written to extract insights

Step 3 – Power BI (Visualization)

- Interactive dashboard created
- KPIs and charts built for business users

5. Python Analysis (Data Cleaning & Feature Engineering)

In this section, the dataset was cleaned and transformed using Python to ensure it is ready for analysis. The process includes data loading, cleaning, handling missing values, and creating new features.

a. Data Loading - The dataset was loaded into Python using the pandas library to begin the analysis.

[1]:
```
##fetch csv file into the notebook
import pandas as pd
df = pd.read_csv(r'C:\Users\pujari madhuri\OneDrive\文档\Pivot Table All Practice Files\customer_shopping_behavior.csv')
```

[2]:
```
##Display the top 5 row present in dataset
df.head()
```

[2]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Paym Met
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Ve
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	(
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Cr (
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	Pa
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	Pa

b. Understanding the Data - To understand the dataset structure, data types, and missing values, initial exploration was performed.

[3]:
```
##Checking total no of rows, column, and data types present in excel dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

```
[4]: ##Summary Statistical analysis of all categories(numerical, and non-numerical data)
     df.describe(include='all')
```

[4]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN

c. Handling Missing Values - Missing values were identified and handled to ensure data accuracy. The missing values in the review_rating column were filled using the median.

```
[5]: ##Checking missing values in respective column
     df.isnull().sum()
```

```
[5]: Customer ID               0
     Age                       0
     Gender                    0
     Item Purchased            0
     Category                  0
     Purchase Amount (USD)     0
     Location                  0
     Size                      0
     Color                     0
     Season                    0
     Review Rating            37
     Subscription Status       0
     Shipping Type             0
     Discount Applied          0
     Promo Code Used           0
     Previous Purchases        0
     Payment Method            0
     Frequency of Purchases    0
     dtype: int64
```

```
[6]: ## 1. Null values are replaced with mean or median
     ## 2. We should choose 'Median' over mean because mean can get affected by outliers and would skew
     ##    distribution of our data.
     ## 3. Median is robust to outliers
     ## 4. As beginner we usually fill all the missing values with overall median of the entire column
     ##   (but this is not best practice)
     ## 5. So considering overall median of the review rating there would be causing a forcing of
     ##    clothing item category to get same review rating as footwear items. And it result Bias into dataset
     ## 6. The Best practice is Input the missing values with the 'median review rating within each category'

     df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

```
[7]: df.isnull().sum()
```

```
[7]: Customer ID               0
     Age                       0
     Gender                    0
     Item Purchased            0
     Category                  0
     Purchase Amount (USD)     0
     Location                  0
     Size                      0
     Color                     0
     Season                    0
     Review Rating             0
     Subscription Status       0
     Shipping Type             0
     Discount Applied          0
     Promo Code Used           0
     Previous Purchases        0
     Payment Method            0
     Frequency of Purchases    0
     dtype: int64
```

d. Column Standardization - Column names were standardized (converted to lowercase and snake_case) to improve readability and consistency.

[8]:
```
## 1. observe above all column names it has Uppercase with lowercase and also spacing between them or even
##    special characters
## 2. And we need to follow certain fromat when using column name in the pyhton or sql code
## 3. The Best practice is to put column names in 'SNAKE CASING'- all lowercase with underscores.

df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ','_')
df.columns
```

[8]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

[9]:
```
## above column purchase_amount_(usd) and replace it as purchase_amount

df= df.rename(columns={'purchase_amount_(usd)':'purchase_amount'})
df.columns
```

[9]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

e. Feature Engineering - Age Group - A new column age_group was created to categorize customers into groups for better analysis.

[10]:
```
## create a column age_group
labels = ['Young Adult' , 'Adult' , 'Middle-aged' , 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4 , labels = labels )
df[['age','age_group']].head(10)
```

[10]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

f. Feature Engineering - Purchase Frequency - A new column was created to analyze how frequently customers make purchases.

[11]:
```
## create a column purchase_frequency_days
frequency_mapping = {
    'Fortnightly' : 14 ,
    'Weekly' : 7,
    'Monthly' : 30 ,
    'Quarterly' : 90,
    'Bi-Weekly' : 14 ,
    'Annually' : 365,
    'Every 3 Months' : 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

g. Remove Redundant Columns - Unnecessary or duplicate columns were removed to simplify the dataset.

```
[12]: df[['discount_applied', 'promo_code_used']].head(10)
```

[12]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[13]: ## lets check whether these above two columns are same , or there may be chances for exculsive customers
      ## company may provide discount without promo_code
      ## lets see which one of the columns redundant

      (df['discount_applied'] == df['promo_code_used']).all()
```

```
[13]: True
```

```
[14]: ## Remove column
      df = df.drop('promo_code_used', axis=1)
```

h. Connecting Jupyter Notebook with PostgreSQL - To establish a connection between Python and PostgreSQL, required libraries were installed and database credentials were configured.

```
[15]: ## move this dataset into Postgress SQL so that we can run SQL queries to do some deeper data analysis
      ## 1. open pgAdmin and create new database 'customer_behaviour'
      ## 2. We need to connect our jupyter notebook with Postgress SQL
      ## 3. And load the data into postgress SQL

      !pip install psycopg2-binary sqlalchemy

      ## psycopg2-binary is the connector
      ## sqlalchemy is the toolkit
```

```
Requirement already satisfied: psycopg2-binary in c:\users\pujari madhuri\appdata\local\programs\python\python310\lib\site-packages (2.9.11)
Requirement already satisfied: sqlalchemy in c:\users\pujari madhuri\appdata\local\programs\python\python310\lib\site-packages (2.0.49)
Requirement already satisfied: greenlet>=1 in c:\users\pujari madhuri\appdata\local\programs\python\python310\lib\site-packages (from sqlalchemy) (3.4.0)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\pujari madhuri\appdata\local\programs\python\python310\lib\site-packages (from sqlalc
hemy) (4.15.0)
```

```
[16]: ## Step 1 : Connect to PostgreSQL
      ## Replace placeholders with your actual details
      username = "postgres" #default user
      password = "root" # the password you set during installation
      host = "localhost" #if running locally
      port = "5432"       # default PostgreSQL port
      database = "customer_behaviour"  # the database you created in pgAdmin

      from sqlalchemy import create_engine
      engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")


      ## Step 2 : Load DataFrame into PostgreSQL

      table_name = "customer" # choose any table name
      df.to_sql(table_name, engine, if_exists="replace", index=False)

      print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
```

```
Data successfully loaded into table 'customer' in database 'customer_behaviour'.
```

6. SQL Analysis (Business Queries & Insights)

After cleaning the dataset in Python, the data was loaded into a PostgreSQL database for structured analysis. SQL was used to answer key business questions related to customer behavior, revenue, and purchasing patterns.

Q1. What is the total revenue generated by male vs. female customers?

```
SELECT gender, SUM(purchase_amount) AS total_revenue FROM

customer

GROUP BY gender

ORDER BY total_revenue DESC;
```

	gender text	total_revenue numeric
1	Male	157890
2	Female	75191

Q2. Which customer used a discount but still spent more than the average purchase amount?

```
select customer_id, purchase_amount from customer where

discount_applied = 'Yes'

and purchase_amount >= (select avg(purchase_amount) from customer)
```

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62

Q3. Which are the top 5 products with the highest average review rating?

```
select item_purchased, round(avg(review_rating::numeric),2) as "Average Product Rating"  from customer

group by item_purchased

order by avg(review_rating) desc limit 5
```

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

Q4. Compare the average Purchase Amounts between Standard and Express Shipping.

```
select shipping_type, round(avg(purchase_amount),2) from customer where

shipping_type = 'Standard' or shipping_type = 'Express'

group by shipping_type
```

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

Q5. Do subscribed customers spend more? Compare average spend and total revenue between subscribers and non-subscribers

```
select subscription_status,

count(customer_id) as total_cuatomers,

round(avg(purchase_amount),2) as "Average spend",

round(sum(purchase_amount),2) as "total revenue" ,

round(SUM(purchase_amount) * 100.0 /

(SELECT SUM(purchase_amount) FROM customer), 2)

AS percentage_of_total from

customer

group by subscription_status

order by "total revenue" ,"Average spend" desc
```

	subscription_status text	total_cuatomers bigint	Average spend numeric	total revenue numeric	percentage_of_total numeric
1	Yes	1053	59.49	62645.00	26.88
2	No	2847	59.87	170436.00	73.12

Q6. Which 5 products have the highest percentage of the purchase with discounts applied?

```
select item_purchased,

round(count(case when discount_applied = 'Yes' then 1 end)*100.0/count(*),2) as  "Discount

 percentage"

from customer

group by item_purchased

order by "Discount percentage" desc limit 5
```

	item_purchased text	Discount percentage numeric
1	Hat	50.00
2	Sneakers	49.66
3	Coat	49.07
4	Sweater	48.17
5	Pants	47.37

Q7. Segement customer into New, Returing, and Loyal based on their total number of previous purchases, and show the count of each segment.

```
select

  case

  when previous_purchases = 1 then 'New'

  when previous_purchases <= 10 then 'Returning' else 'Loyal'

  end

  as "Customer tag",

  count(*) as "number of customers" from

customer

group by "Customer tag"

order by "number of customers"

    --   or --
```

```
with customer_type as (
select customer_id, previous_purchases, case
when previous_purchases = 1 then 'New'
when previous_purchases between 2 and 10 then 'Returning' else 'Loyal'
end as "Customer Segment" from
customer
)
select "Customer Segment" , count(*) as "Number of Customers" from
customer_type
group by "Customer Segment"
order by "Number of Customers"
```

	Customer tag text	number of customers bigint
1	New	83
2	Returning	701
3	Loyal	3116

Q8. What are the top 3 most purchased products within each category?

```
select rank_number,category, item_purchased,
purchase_count
from
(
    select category, item_purchased, count(*) as purchase_count,
        row_number() over (partition by category order by count(*) desc) as
        rank_number
    from customer
    group by category , item_purchased
```

```
)
where rank_number <= 3
-- or --
with item_count as (
    select category, item_purchased, count(customer_id) as total_orders, row_number() over
        (partition by category order by count(customer_id) desc) as rank_number
    from customer
    group by category, item_purchased
)

select rank_number, category, item_purchased, total_orders from
item_count
where rank_number <= 3
```

	rank_number bigint	category text	item_purchased text	purchase_count bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

Q9. Are customer who are repeat buyers (more than 5 previous purchases) also likely to subscribe?

```
select subscription_status, count(customer_id)

from customer

where previous_purchases > 5 group

by subscription_status
```

	subscription_status text	count bigint
1	No	2518
2	Yes	958

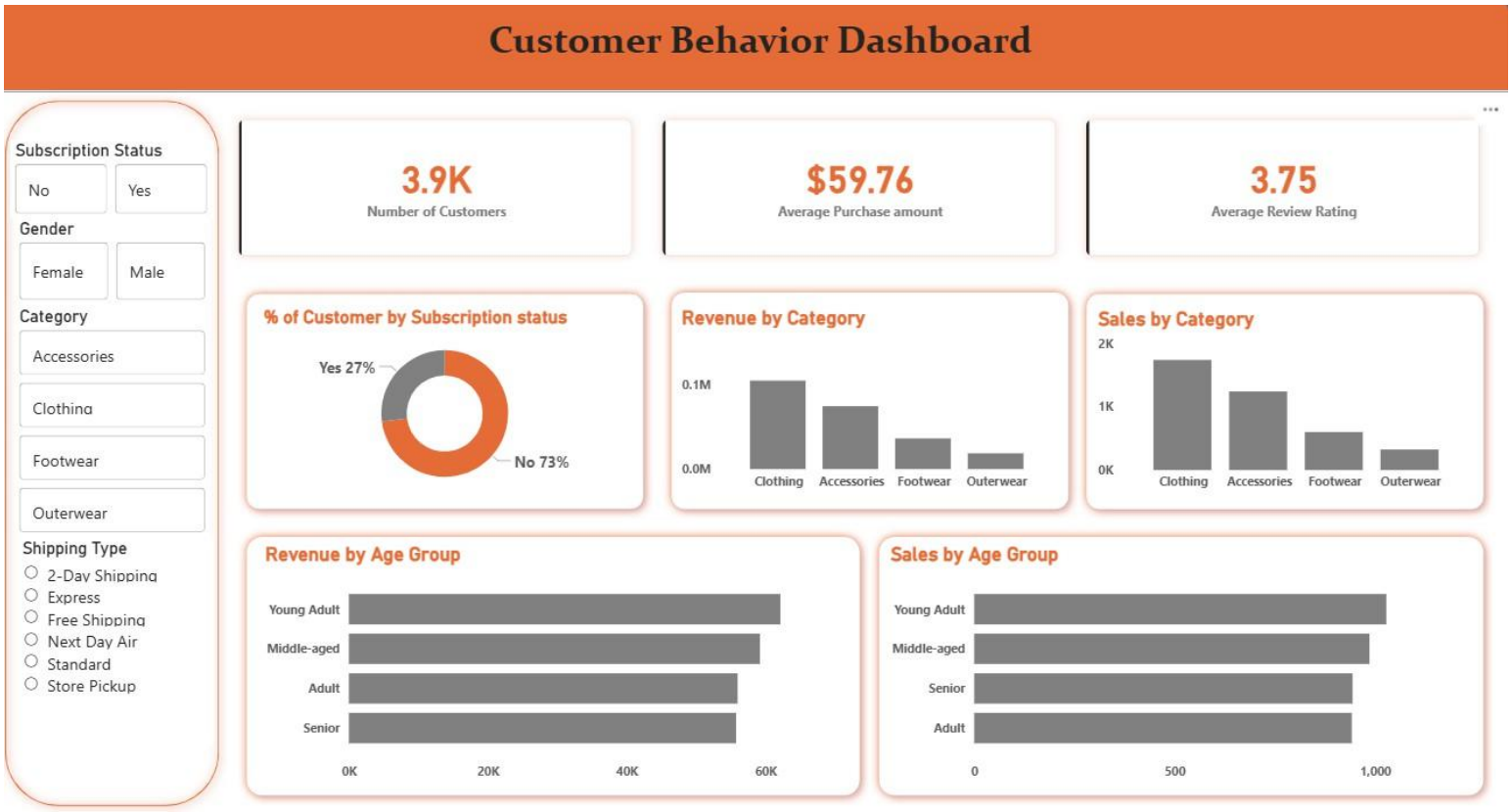
Q10. What is the revenue contribution of each age group?

```
select age_group,sum(purchase_amount) as total_revenue from

customer

group by age_group

order by total_revenue desc;
```

	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

7.Power BI Dashboard

Finally, we built an interactive dashboard in Power BI to present insights visually.



8. Key Insights

- Subscribers contribute significantly higher revenue
- Repeat customers are the core revenue drivers
- Certain categories consistently outperform others
- Discount usage influences purchase behavior
- High-value customers belong to specific age groups

9. Business Recommendations

- Introduce loyalty programs for repeat customers
- Promote subscription benefits
- Optimize discount strategies to protect margins
- Focus marketing on high-performing categories
- Target high-value customer segments